BA TASK 4 – High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

Objective

To build a frequent pattern mining engine from scratch using a compressed FP-Tree. The system stores transactional data efficiently, uses node-link pointers, constructs conditional FP-Trees recursively, and generates high-confidence association rules.

Problem Statement

Given a large collection of customer transactions, identify frequent item combinations and generate useful association rules while reducing unnecessary candidate generation and maintaining efficient execution.

Approach

1. Read the transaction dataset.
2. Count the frequency of each item.
3. Remove items below the minimum support threshold.
4. Sort remaining items by descending frequency.
5. Insert transactions into the compressed FP-Tree.
6. Maintain node-link pointers for equal items.
7. Build conditional pattern bases and conditional FP-Trees recursively.
8. Extract frequent patterns and generate association rules.

FP-Tree Structure

An FP-Tree contains a root node, item nodes, parent-child relationships, item counts, and node-link pointers. Common prefixes of transactions share the same tree path, reducing memory usage.

Node Link Pointers

For each item, a header table stores a link to its first occurrence in the tree. Each occurrence is connected to the next occurrence of the same item. This allows the mining algorithm to quickly traverse all nodes representing an item.

Conditional FP-Tree

For a selected item, its linked nodes are traced upward to the root to form conditional pattern paths. These paths are filtered using minimum support and inserted into a new conditional FP-Tree. The process is repeated recursively to discover frequent patterns.

Association Rules

For a frequent itemset X, a rule $A \rightarrow B$ can be generated when $A \cup B = X$. Confidence is calculated as: Confidence($A \rightarrow B$) = Support($A \cup B$) / Support(A). Rules whose confidence is at least the selected threshold are retained.

Sample Python Implementation

```
from collections import Counter

class Node:
    def __init__(self, item, count=1, parent=None):
```

```
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None

def count_items(transactions):
    freq = Counter()
    for t in transactions:
        freq.update(t)
    return freq

def insert_tree(items, root):
    current = root
    for item in items:
        if item in current.children:
            current.children[item].count += 1
        else:
            current.children[item] = Node(item, 1, current)
        current = current.children[item]

transactions = [
    ["milk", "bread", "eggs"],
    ["bread", "butter"],
    ["milk", "bread", "butter"],
    ["bread", "eggs"]
]

freq = count_items(transactions)
root = Node("ROOT", 0)

for t in transactions:
    ordered = sorted(t, key=lambda x: (-freq[x], x))
    insert_tree(ordered, root)

print("Frequent items:", freq)
print("FP-Tree constructed successfully.")
```

Sample Output

```
Frequent items: Counter({'bread': 4, 'milk': 2, 'eggs': 2, 'butter': 2})
FP-Tree constructed successfully.
```

Time Complexity

Frequency counting requires O(N) item visits, where N is the total number of items in all transactions. FP-Tree construction is approximately O(N) after frequency ordering. Mining complexity depends on the number and density of conditional pattern bases.

Conclusion

The compressed FP-Tree approach efficiently represents transactional data by sharing common prefixes and avoids the repeated candidate-generation process used by traditional Apriori-style mining. Node links and recursive conditional FP-Trees support efficient frequent-pattern discovery.